Embedded Systems Documentation D

Landry Ellis

June 2026

1 2 Wheel Balance Robot

1.1 Introduction

This project focused on the design and development of a two wheel balancing robot. The goal was to create the system from the ground up rather than copying an existing design. This included designing the chassis, creating a custom PCB, selecting the required sensors and drivers, assembling the board, and preparing the code structure needed for balance control.

The robot was designed around two DC motors, a motor driver, an IMU, magnetic encoder feedback, an RGB status LED, and a WiFi communication module. The WiFi module was included so that the robot could eventually be controlled and tuned wirelessly. Since the robot needs to balance on two wheels, the main design challenge was creating a system that could read angle and motion data quickly enough to correct the motor output in real time.

1.2 Design Stage

The first part of the project was the system-level design. The main blocks of the robot were the microcontroller, motor driver, motors, IMU, encoder, RGB LED, and ESP8266 communication module.

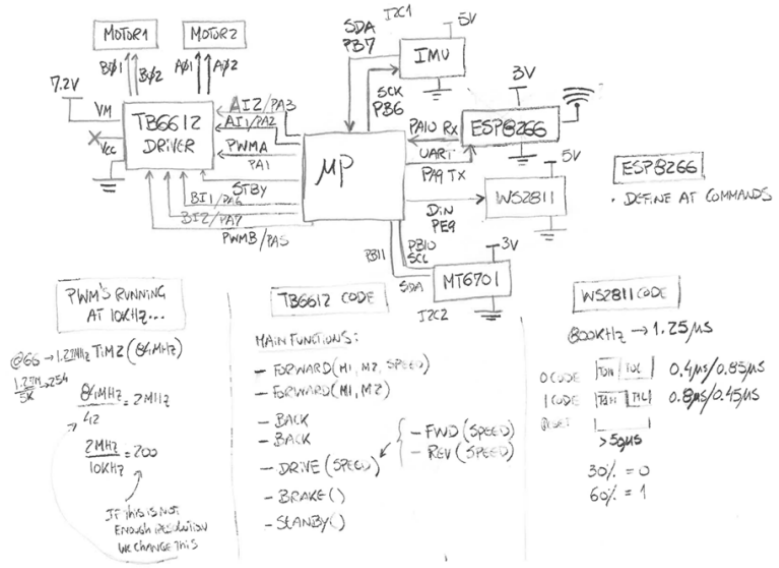

Figure 1: System block diagram for the two wheel balance robot.

The minimum system required two motors, one motor driver, one IMU, one magnetic encoder, one RGB LED, and one WiFi communication module. The motor driver selected for the design was the TB6612. The IMU selected was the MPU6050. The magnetic encoder selected was the MT6701. The communication module selected was the ESP8266.

The ESP8266 was intended to act as a wireless interface between the user and the robot. The ESP8266 would host a WiFi access point and web server. A user would connect a laptop, phone, or tablet to the access point, open the robot control page in a browser, and send commands to the robot. The ESP8266 would then communicate with the main microcontroller over UART.

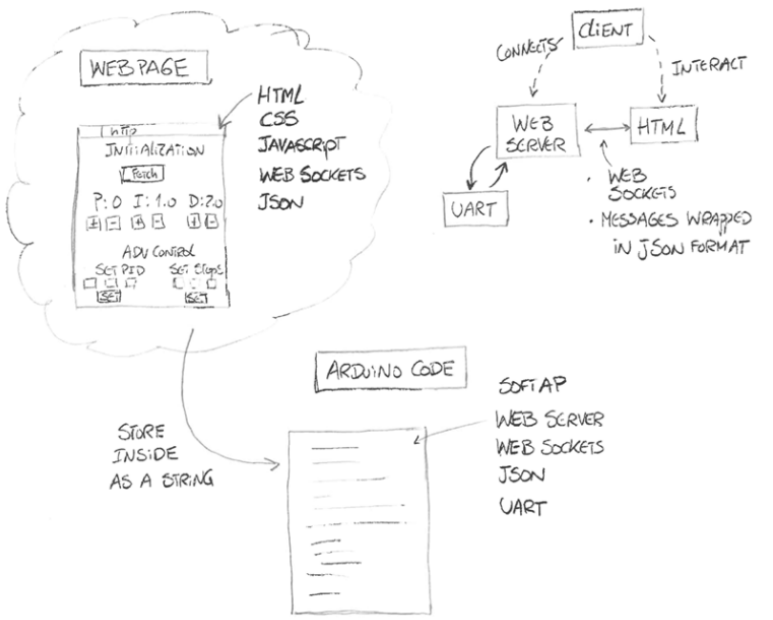

Figure 2: Planned ESP8266 web interface and UART communication structure.

The main microcontroller selected for the custom PCB was an STM32F401RBT6 in an LQFP package. The pins were selected so that the board could support the required timers, PWM outputs, I2C communication, UART communication, encoder inputs, and general GPIO signals.

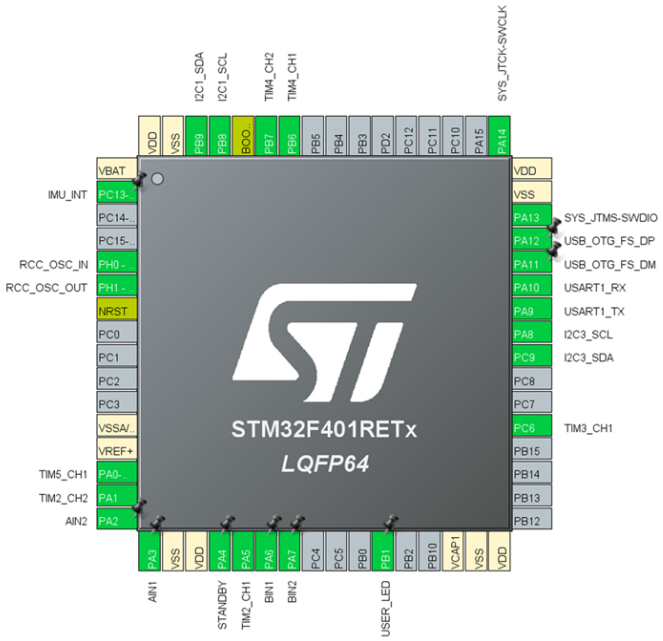

Figure 3: STM32 pinout used for the custom PCB design.

The software structure was also planned during the design stage. The main code was intended to manage motor PWM, encoder readings, IMU measurements, RGB LED updates, and UART communication with the ESP8266.

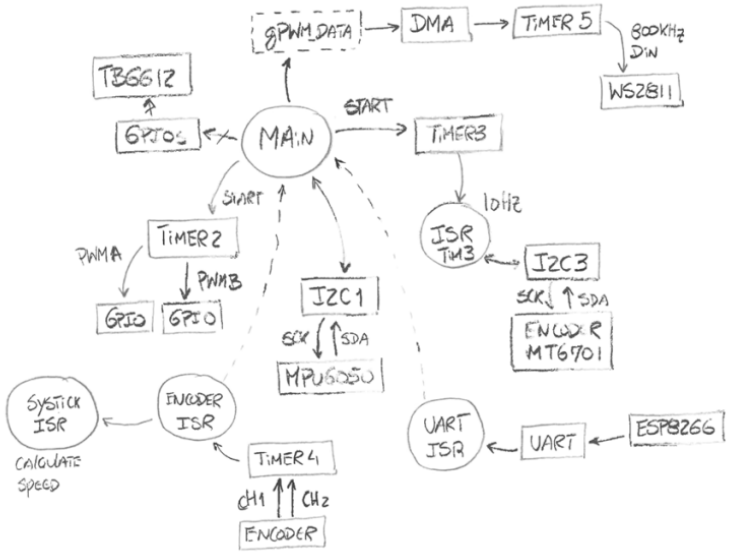

Figure 4: Planned code structure for the robot firmware.

1.3 Prototype Stage

N/A. This was done by Dr. Martins before the beginning of this class.

1.4 PCB Design

After the system design and prototype testing, the custom PCB was designed. The schematic included the STM32 microcontroller, motor driver, voltage regulators, IMU, encoder connector, ESP8266 connector, RGB LED circuit, SWD programming header, reset and boot circuitry, and power connections.

1.4.1 Schematic

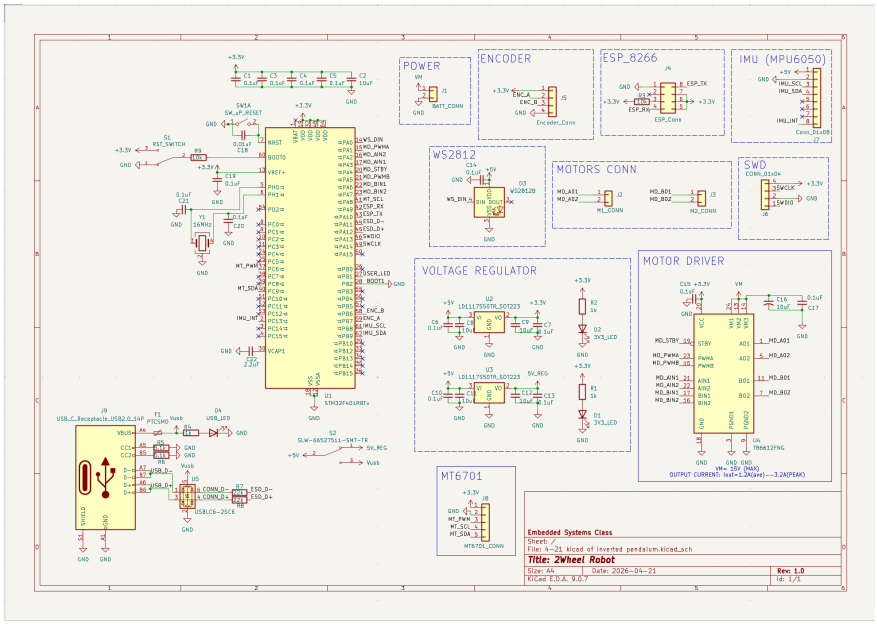

Figure 5: Full schematic for the custom two wheel balance robot PCB.

Note: This PCB layout isn't entirely correct. One voltage regulator LED should be connected to 5V_REG. There are also capacitor values that are incorrect and one footprint is the small size surface mount, but should be the large size.

The correct Schematic can be found here: http://engredu.com/2026/05/01/2-wheel-balance-robot/

1.4.2 Layout

The PCB layout was created to keep the board compact while still allowing access to the main connectors and programming pins. The motor driver and power components were placed near the motor and battery connections. The microcontroller was placed near the center of the board so that routing to the peripherals could be managed more easily.

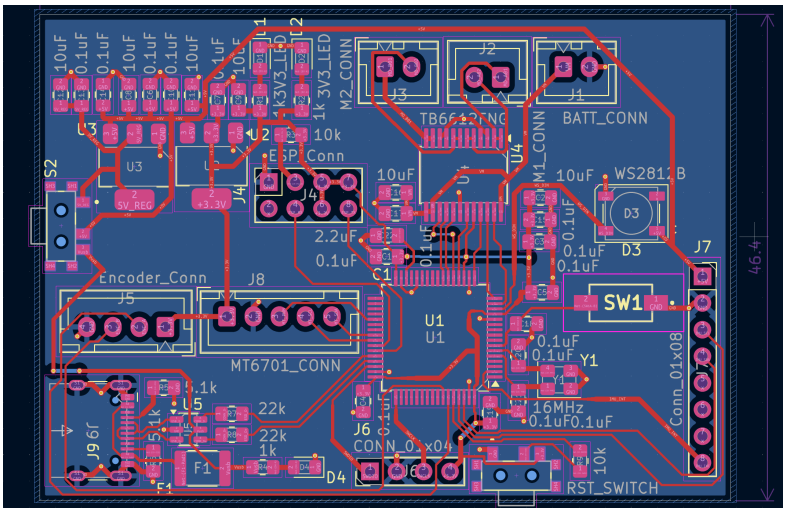

Figure 6: PCB layout for the custom robot controller board.

1.4.3 3D Rendering

A 3D rendering was created to check the physical layout of the board, connector positions, and general component placement before manufacturing.

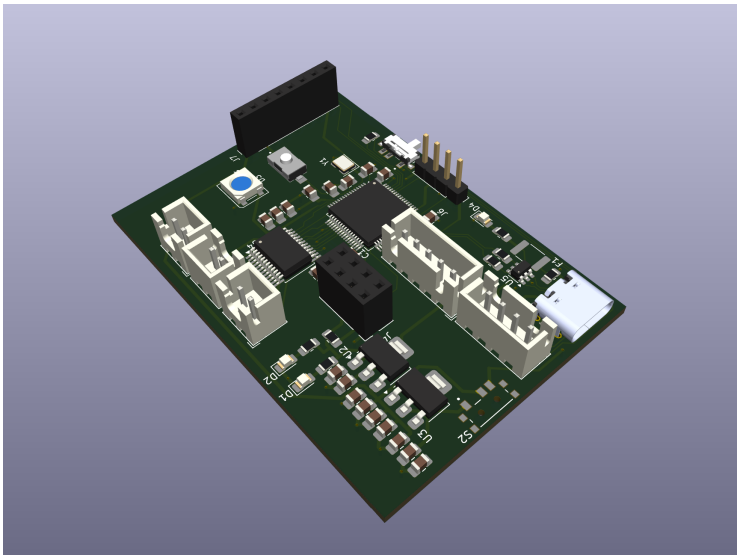

Figure 7: 3D rendering of the custom robot PCB.

1.5 Assembly Stage

The custom PCBs were manufactured and assembled in house. Unlike the reference build, solder paste and a stencil were not used for the assembly of our boards. This was because each student designed a unique PCB, so individual stencils could not be made for every board. Instead, the boards were assembled using normal hand soldering.

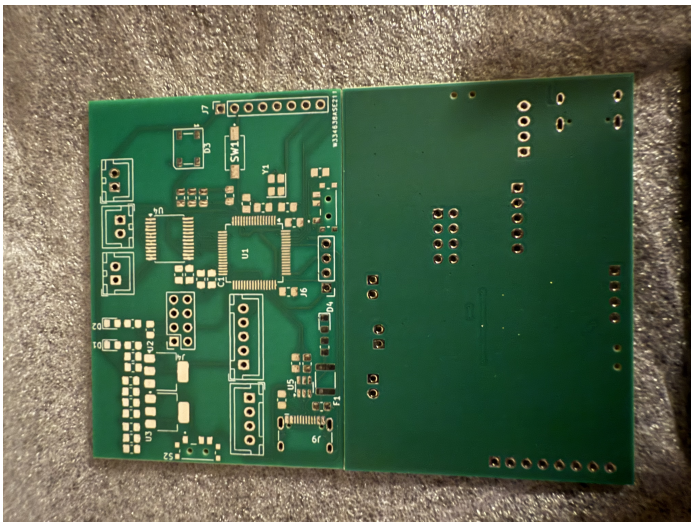

Figure 8: Manufactured custom PCB before assembly.

Most of the components were soldered to the board by hand. The smaller and more difficult components were the most challenging to attach cleanly, especially the motor driver and the STM32 microprocessor. These two parts were first placed on the board by hand, but they likely had bridged connections because of the small pin spacing.

After the initial hand soldering, Dr. Martins helped complete the motor driver and microprocessor connections using a better soldering iron. This step was necessary because those components required more precise soldering than the rest of the board.

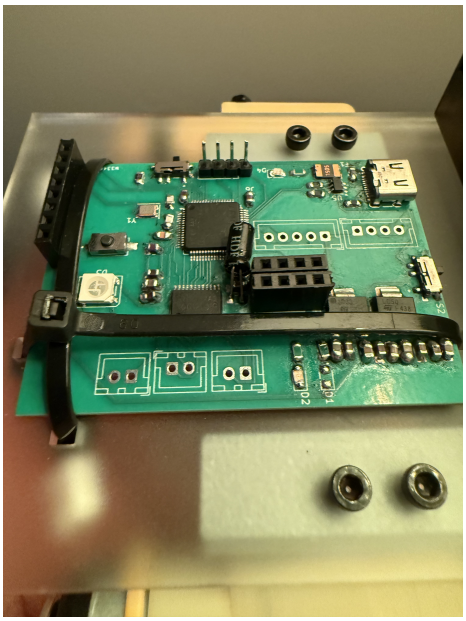

Figure 9: PCB after soldering corrections.

The first planned firmware test was a basic blink program. This test was supposed to confirm that the microcontroller could be programmed and that the board was able to run simple code. However, the blink test could not be completed because communication through the USB port was not working. The cause of the USB communication issue is unknown.

Because the board could not communicate properly over USB, the project could not move into the later testing steps that required a laptop connection to the board. This prevented full verification of the ESP8266 interface, telemetry, remote control, and final balance control code.

1.6 3 Wheel Robot Test Platform

The next planned stage was to test the control electronics on a three wheel robot platform before attempting full two wheel balancing. The goal of this stage was to verify that the PCB, motors, encoder, IMU, ESP8266, and RGB LED could all work together before adding the difficulty of balancing control.

1.6.1 Chassis

The three wheel chassis was designed so that the PCB, motors, wheels, and battery holder could be mounted securely. The chassis was intended to be laser cut from 3 mm acrylic, with M3 hardware used for the mounting holes.

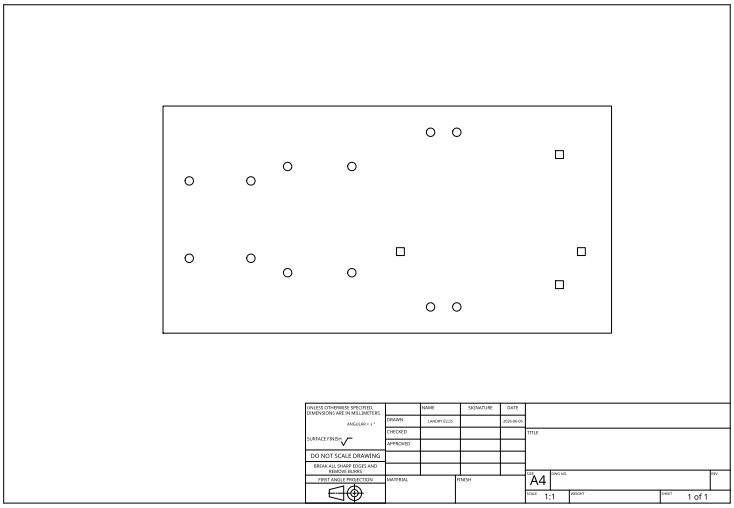

Figure 10: Planned chassis layout for the three wheel robot test platform.

1.6.2 Communication

The ESP8266 was intended to be used as a remote web server for controlling the robot. The user would connect to the ESP8266 access point and send commands from a browser. The ESP8266 would then send those commands to the STM32 over UART.

The planned UART command structure was:

- `w` – move forward
- `s` – move backward
- `a` – move left
- `d` – move right
- `x` – stop

The planned telemetry packet format was:

```
$speed,roll,pitch,counter\n
```

This telemetry format was intended to send basic robot data back to the web interface, including motor speed, IMU roll, IMU pitch, and a counter value.

1.6.3 Robot Interaction

The next step would have been to connect a laptop, phone, or tablet to the ESP8266 WiFi access point. After connecting to the robot network, the user would open a browser and enter the ESP8266 IP address to access the web control interface.

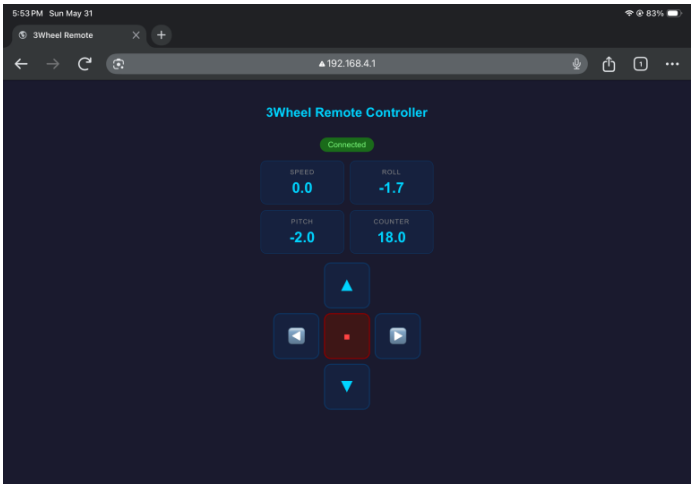

Figure 11: Planned browser-based robot control interface.

This stage was not completed because of issues with the custom PCB. The robot was not successfully tested through the laptop-to-board connection, so the web interface, live telemetry, and remote driving behavior were not verified on the final hardware.

1.7 2 Wheel Balance Robot

The final stage of the project was intended to use the completed electronics and chassis to run the robot as a true two wheel balancing system. This stage would have combined IMU angle measurements, encoder feedback, and motor control to keep the robot upright.

The planned control approach was to use the IMU to estimate the robot angle and use the motor outputs to correct the robot when it tilted forward or backward. The encoder feedback would provide wheel motion information, which could be used to improve stability and control the robot's movement.

This stage was not reached because the PCB issues prevented full communication and drive testing. As a result, the balancing algorithm was not fully implemented or tested on the completed robot hardware.

1.8 GitHub Repository

The project files can be found here:

- https://github.com/LandryEllis/ENCE_3231/tree/main/2WheelBotFinal5-5
- Note: Due to metadata weirdness with the compiled pdf from overleaf this link will not copy paste the underscore correctly. This folder is near this PDF's location in the GitHub.

1.9 References

Reference build:
http://engredu.com/2026/05/01/2-wheel-balance-robot/

2 Troubleshooting

- The first thing done during the troubleshooting phase was try to connect to the board using a different USB C cable that had worked with other boards.
- Then the board was connected to other laptops which had worked with other boards.
- The board was then probed with a multimeter to check all PCB connections. All connections seemed to carry the signals to the correct nodes.

2.1 Speculation

- The differential pair from the USB to the microprocessor could be too long.
- The microprocessor got burned while soldering on day one.
- A solder connection to a decoupling capacitor or another connection is poor. The node shows voltage with the multimeter but introduces unwanted resistance or capacitance. Additionally, pressing the multimeter probe on this connection could temporarily connect the circuit and deceive the user.